

Orchestras Not Factories: Stickfo 原则下 的人机协作工作系统

SSS & Codex

2026 年 7 月 11 日



视频作者/频道: AI Engineer / WF2026

发布日期: 2026 年 7 月 1 日

视频时长: 约 17 分钟 (主题切片约 1024 秒; 用户边界为 1019 秒)

视频链接: <https://www.youtube.com/watch?v=htM02KMNZnk&t=9639s>

PDF 制作 Skill: <https://github.com/wdkns/wdkns-skills>

目录

1 人口：编程智能体把“个人终端”变成“团队工作面”	2
1.1 动机：并行度增加以后，终端不再是唯一问题	2
1.2 观点：统一界面把一组执行单元变成可观察的工作面	2
1.3 机制与证据：从观察优秀构建者到提出六条原则	3
1.4 局限：产品入口、经验原则与可迁移结论需要分层	4
1.5 本节小结	4
2 原则一：靠近技术前沿	5
2.1 动机：工作方式的变化快于组织的信息传递	5
2.2 观点：前沿接触会暴露新的产品机会	5
2.3 机制与证据：从仓库并行到内部工具	5
2.4 组织迁移：非创业团队需要一个前沿角色	6
2.5 局限：near 与 at 的边界	6
2.6 本节小结	6
3 原则二——不要试图打败市场	6
3.1 先问：为什么它还不是默认能力？	7
3.2 Ralph 循环：普适工作流不必被过度定制	7
3.3 有效市场假说只是解释性类比	7
3.4 真实业务优势信息不是金融收益指标	7
3.5 长对话 React 性能：值得优化的真实案例	8
3.6 把原则转成工程投入判断	8
3.7 本节小结	8
4 原则三——建立人工审查保留区	8
4.1 人工审查保留区是风险分区	8
4.2 migrations 的 CI 规则：把保留区落成硬约束	9
4.3 治理保留区与内容质量判断：证据层级不能混用	9
4.4 Slack、CLAUDE.md 与 skills：高杠杆上下文也需要人工打磨	9
4.5 从保留区回到整体工作流	10
4.6 本节小结	10
5 原则四：持续供给上下文——让组织事实成为智能体可检索的基础设施	10
5.1 从启动规则到组织记忆	10
5.2 CIA：把团队活动汇入一个集中位置	11
5.3 SQL tool：把集中上下文变成可取用接口	11
5.4 部署时仍需补足治理条件	12
5.5 本节小结	12
6 原则五：自由运行智能体——把执行从笔记本寿命中解放出来	12
6.1 定义：自由运行不是无边界权限	13
6.2 从 Git 工作树到云端沙盒	13
6.3 实时协作工作区：从个人任务到团队状态	14

6.4	外部消息如何启动一个工作区	15
6.5	部署时仍需补足治理条件	16
6.6	本节小结	16
7	原则六：乐团式协作，而不是工厂式流水线	16
7.1	问题：自动化提高产量，却可能改变人的位置	17
7.2	核心画面：人挥动指挥棒，智能体团队展开工作	17
7.3	为什么工厂式 feature 管理不足以承载新工作方式	18
7.4	工具建设者的责任：让人感觉处于其中	18
7.5	本节小结	19
8	总结与迁移：把 Stickfo 组合成一套工作系统	19
8.1	讲者的结尾：从六条原则回到 Stickfo	19
8.2	六条原则的系统链条	19
8.3	迁移检查表：把原则变成设计问题	20
8.4	讲者愿景与教学归纳的分层	20
8.5	本节小结	21

全文中心结论

AI 编程工具的高效组织方式，核心是建立“人处于创造中心、智能体组成可编排乐团”的工作系统：靠近前沿以发现机会，用市场启发式控制通用 workflow 投入，在关键区域保留严格人工审查，把组织上下文持续供给给智能体，让智能体在持久云端沙盒中运行与协作，最终把自动化能力用于扩大人的创造流，而不是把人变成流水线经理。

来源与阅读边界

本笔记只对主题 13 的英文原始字幕、英文事件和主题视频切片负责。时间戳使用原视频绝对时间‘02:40:39–02:57:38’；末尾致谢与离场话术不进入教学正文。个别专名存在字幕识别差异，以画面和语义校准；演讲未展开权限、审计、成本与失败恢复，这些属于部署时仍需补足的工程条件。‘Stickfo’是讲者给出的记忆标签，来源没有可靠提供逐字母展开。

1 人口：编程智能体把“个人终端”变成“团队工作面”

本节的中心判断是：Conductor 的产品入口揭示了一个工作对象的变化。开发者开始同时管理一组编程智能体，问题随之从“如何打开一个终端”扩展为“如何安排多个执行单元、保持共享上下文、观察进展并作出取舍”。因此，Conductor 在本主题中首先承担案例入口的作用；它帮助读者看见多智能体工作面正在形成，随后再进入六条原则所回答的组织问题。

本节主张

Conductor 的关键价值可以先理解为一个统一的编排入口：它把原本分散在多个终端窗口中的编程智能体放到同一工作面上。这个入口的教学意义，在于把“并行执行”转化为“工作系统设计”问题；六条原则正是对观察、投入、审查、上下文、运行空间和人机关系的整体回答。

1.1 动机：并行度增加以后，终端不再是唯一问题

传统的单人开发流程通常围绕一个终端、一个编辑器和一个当前任务展开。当多个编程智能体 (coding agents) 同时参与代码库探索、修改或执行任务时，开发者需要同时面对多个运行状态。每个终端窗口都可能承载不同的任务、上下文和待审查结果，人的工作因此出现了新的协调负担。讲者在介绍 Conductor 前先交代自己作为联合创始人身份，并把听众带入人工智能工程的讨论场景。视频绝对时间：02:41:13–02:41:23。

这里的关键变化在于：窗口数量增加以后，开发者还需要知道每个智能体正在做什么、它们是否共享同一组事实、哪些结果应当合并，以及哪些修改需要人先看。于是，速度问题开始与工作空间、上下文、审查和协作问题耦合起来。这也是本节把 Conductor 作为“团队工作面”来解释的原因：读者需要先理解协调对象发生了变化，才能理解后续原则为何延伸到产品功能以外。

1.2 观点：统一界面把一组执行单元变成可观察的工作面

讲者将 Conductor 定义为一个同时管理一组编程智能体的桌面应用。传统做法需要为不同的云端代码工具、代码执行工具或其他编程智能体打开许多终端窗口；Conductor 提供一个统一界面来管理它们。这个统一入口的教学含义超过窗口收纳：它至少包含三个面向人的动作，把多个任务放

到同一视野中，持续观察各个执行单元的状态，以及在需要时对任务进行取舍和重新安排。视频绝对时间：02:41:23-02:41:40。

从读者视角看，这个入口可以用一个简单的层级来理解：终端是执行通道，智能体是执行单元，统一界面是人的编排面。执行通道负责运行命令和产生修改，智能体负责围绕任务进行探索与行动，编排面负责让人保持全局判断。三者分开以后，多智能体系统的设计重点就从“能否启动更多任务”转向“人能否持续理解并控制任务集合”。

人口案例的阅读方式

读者可以把 Conductor 看成一面控制台。控制台解决的是多执行单元的可见性与组织入口；六条原则继续追问组织应当观察什么、把时间投入在哪里、哪些区域保留人工判断，以及最终希望人处于怎样的工作状态。

1.3 机制与证据：从观察优秀构建者到提出六条原则

讲者给出的第二层证据来自长期观察。构建 Conductor 的过程中，他近距离观察了许多优秀构建者的工作方式，记录他们实际会做什么、主动避免什么，再把这些重复出现的行为整理为一组原则。这个叙述把产品入口和工作方法连接起来：Conductor 提供观察多智能体工作的场景，优秀构建者的行为则提供抽象原则的经验来源。视频绝对时间：02:41:40-02:41:58。

在画出 Conductor 界面之后，讲者把六条原则定位为“成为组织中最快构建者”的方法入口。这里的“最快”应理解为组织内创造与交付能力的综合速度，不能直接理解为未经验证的性能排名。六条原则首先形成一张问题地图：

- 观察新能力与新工作方式，及时识别可能改变交付方式的变化。
- 判断通用工作流优化的投入边界，把时间留给真正有组织信息优势的问题。
- 划定必须由人严格审查的代码、文档和规则区域。
- 持续向智能体提供组织事实、讨论和任务上下文。
- 提供能够支持持续运行、探索和协作的执行空间。
- 让人保留方向、取舍与整体编排责任，并维持创造性的工作体验。

这张问题地图的价值在于分离了几个经常被混为一谈的层面。六个问题依次落在观察与发现、投资边界、治理边界、信息供给、运行与协作基础设施，以及人的最终体验上。它们共同构成从外部探索到内部工作关系的递进结构，读者可以据此理解后文每条原则承担的独立职责。讲者在 02:42:02 左右展示界面，并在 02:42:10-02:42:15 将其连接到这些原则的入口。视频绝对时间：02:41:58-02:42:15。

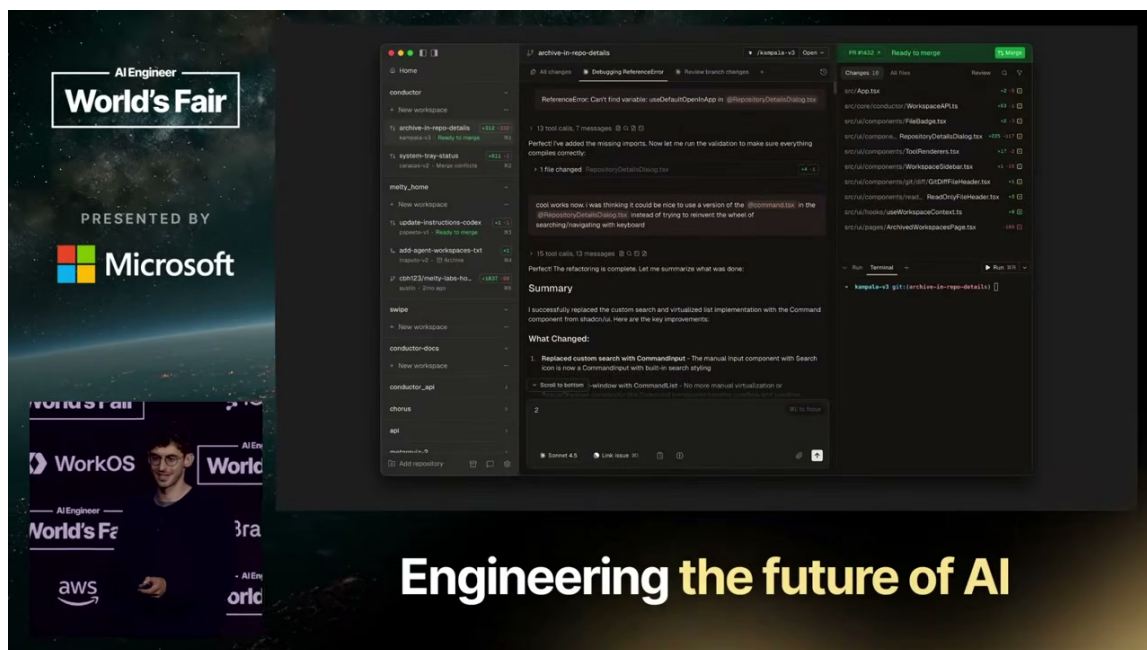


图 1: Conductor 入口画面：多个 workspace、agent 对话、变更文件与终端被放在同一工作面。¹

1.4 局限：产品入口、经验原则与可迁移结论需要分层

本节材料足以说明 Conductor 的入口形态和讲者如何从使用经验提出原则，尚不足以证明统一界面必然提升所有团队的交付效率。Conductor 是演讲时点上同时管理一组编程智能体的桌面应用；“优秀构建者的工作方式”是讲者的经验性观察；六条原则适合作为可迁移和检验的设计问题。

这一区分也限制了本节的推论范围。统一界面能够改善人的观察入口；上下文缺失、审查责任不清和任务边界混乱仍需后续章节分别处理。本节的职责是建立问题地图，为后续分析留下清晰边界。

不要把入口等同于答案

多窗口被统一管理，只说明协调界面发生了变化。真正的组织能力还取决于信息是否可获得、风险是否被分区、运行空间是否持久，以及人是否仍然负责方向和取舍。后文六条原则分别补足这些条件。

1.5 本节小结

Conductor 的入口把“一个人操作一个终端”的开发画面扩展为“一个人编排一组编程智能体”的团队工作面。讲者从统一界面出发，转而总结优秀构建者的工作习惯，并将其整理为六个互补问题：观察新能力、控制通用优化投入、保留人工审查、供给组织上下文、提供持续运行空间，以及让人承担最终编排责任。读者接下来应把 Conductor 当案例入口，把六条原则当作可迁移的工作系统检查框架。

¹ 视频画面时间区间：02:42:02–02:42:15。

2 原则一：靠近技术前沿

本节的中心判断是：靠近技术前沿（stay near the frontier）的价值，在于持续接触刚出现且可能改变工作方式的能力，并把这些接触转化为真实工作中的新洞见和产品机会。原则中的“靠近”保留了交付工作的重心；它要求组织保持敏锐，也要求组织避免把全部时间消耗在工作流试验本身。

本节主张

“靠近技术前沿”的判断标准是：新工具或新工作流是否让真实工作产生新的工作方式、约束或产品机会。价值落在试验与交付之间形成的可复用洞见上；工具尝试的时间先后本身不足以构成价值。

2.1 动机：工作方式的变化快于组织的信息传递

讲者把这一原则定义为：新事物基本在出现当天就尝试。为说明其具体含义，讲者举出 Ultra Code 和 slashgo 等刚出现的工具，强调持续接触最新能力。对组织而言，这种做法回应了一个现实问题：如果新工作流的变化速度超过了同事网络和常规信息传递的速度，等待经验自然扩散就会使组织在真正采用之前已经落后。视频绝对时间：02:42:15-02:42:39。

这里的动机在于缩短“能力出现”到“组织理解”的距离。新工具只有进入真实任务，才会暴露它改变了哪些操作、增加了哪些协调要求、消除了哪些旧限制。读者可以把前沿尝试看成一项带有交付约束的侦察工作：试验必须不断回到真实代码、真实用户和真实团队协作中接受检验。

2.2 观点：前沿接触会暴露新的产品机会

对创业团队而言，讲者把靠近前沿与发现产品方向直接联系起来。Conductor 团队当时正在构建另一款应用 Chorus，随后因为高频使用一类编程工具而重新观察自己的工作流。这个变化说明，新能力带来的机会可能首先表现为“团队开始以新的方式工作”，随后才表现为一个可以独立交付的产品方向。视频绝对时间：02:42:39-02:42:55。

这一点可以分成两个连续步骤。第一步是把新工具用于足够真实、足够频繁的任务，让瓶颈从偶然体验中浮现出来。第二步是把瓶颈转化为工作流设计：团队发现需要同时处理多个仓库副本和并行任务，于是开始寻找更适合的工作组织方式。前沿接触在这里承担的是发现机制，产品决策仍然需要结合实际任务和团队判断。

2.3 机制与证据：从仓库并行到内部工具

讲者回顾了一个具体演化过程：团队先把代码仓库克隆五次，随后发现工作树，再逐步把这些需求组合成 Conductor 这一内部工具。这个过程展示了“靠近”如何产生洞见。最初的动作服务于多个智能体并行工作；随着重复操作出现，团队意识到仓库副本、任务状态和并行执行需要更统一的组织方式。最终形成的工具根源在持续使用中的摩擦，脱离交付场景的抽象设计无法单独说明这条产品路径。视频绝对时间：02:42:55-02:43:11。

这条证据还给出一个产品机会的识别路径：

1. 先把新能力放入正在交付的工作中。
2. 记录重复出现的操作、等待和协调摩擦。
3. 判断摩擦是否反复影响多个任务，区分稳定瓶颈与一次性偶然问题。

4. 把稳定出现的摩擦提升为 workflow 或产品设计问题。

团队在高频使用新工具时经历了多次仓库复制，发现工作树，随后逐步构建出内部工具。这支持“真实使用能够暴露机会”这一经验判断，尚未证明同样路径适用于所有组织。

2.4 组织迁移：非创业团队需要一个前沿角色

讲者进一步区分了创业公司和普通公司。没有在构建新产品的组织，也应当有一名成员持续掌握最新 workflow，并把理解带回团队。过去，组织可以依赖社交网络和经验自然传递；讲者认为当前变化速度已经让这种传递方式变慢，等待信息逐层扩散可能使团队落后三到六个月。视频绝对时间：02:43:13-02:43:37。

这条建议的组织含义是设置一个“前沿观察角色”，让其他成员获得经过筛选和翻译的判断。该角色需要把试验结果翻译成团队能使用的判断：哪些能力改变了实际任务，哪些只是短期噪声，哪些摩擦值得进入内部工具或流程设计。这样，前沿接触才会从个人兴趣转化为组织学习。

2.5 局限：near 与 at 的边界

near 是本原则中最需要保留的边界词。靠近技术前沿意味着新能力与真实交付之间保持高频接触；*at* 可以理解为把前沿试验本身变成主要工作对象。两者的差别落在时间分配和价值验证上：*near* 仍然让真实任务检验试验，*at* 容易让 workflow 改造取代实际工作。

在 02:43:37，讲者再次强调 *near* 这个词的重要性：前沿接触应当服务于新洞见和真实交付，不能用“尝试得最早”替代“产生了可用认识”。关于 *at* 所带来的具体风险，紧接其后的语义段会继续展开。视频绝对时间：02:43:37。

前沿试验的停止条件

当试验开始持续消耗主要工作时间，却没有暴露新的产品机会、workflow 约束或可复用经验时，组织就需要重新检查自己是在靠近前沿，还是已经把前沿本身当成了项目。

2.6 本节小结

靠近技术前沿是一种连接机制：它把新能力带入真实工作，再把真实工作中的摩擦转化为 workflow 和产品洞见。Conductor 团队从 Chorus 方向转入高频使用新工具，经历多次仓库复制、发现工作树并形成内部工具，说明产品机会可能在前沿实践中逐步显现。对普通组织而言，可以由一名成员承担持续观察与知识回传；判断标准始终围绕新实践是否改善真实工作，尝试时间的早晚属于次要指标。*near* 保留了交付重心，*at* 则提醒读者警惕试验吞噬工作本身。

3 原则二——不要试图打败市场

本节的结论是：通用 workflow 最终可能成为工具的默认能力，组织不必为它投入大量专门优化；真正值得深挖的，是组织掌握而模型通常不知道的用户约束、产品场景与代码库事实。讲者把这个投入边界概括为“不要试图打败市场”（don't try and beat the market）。（原则表述来源时间：02:43:54-02:44:08）这条原则约束的是 workflow 优化的投入强度，不是停止尝试新工具。

3.1 先问：为什么它还不是默认能力？

“靠近技术前沿”解决的是发现问题：开发者需要尽早接触新的工具和工作方式。随之而来的第二个问题是投入判断：当某种工作流突然流行时，是否应当立刻围绕它重写自己的开发流程？讲者给出的启发式问题是：为什么这个工作流还没有成为默认配置？（来源时间：02:43:37–02:44:15）

这个问题把“新鲜感”与“专属价值”分开。若某种做法对几乎所有使用者都有效，它具有普适性；普适能力更可能被模型提供方或默认工具框架吸收。围绕它建立一套复杂的内部流程，可能只是在等待上游把同一能力做成默认功能。

3.2 Ralph 循环：普适工作流不必被过度定制

讲者以 Ralph 循环作为例子。当 Ralph 循环变得流行时，问题不是“它是否值得了解”，而是“是否值得花大量时间把自己的工作流专门优化成适配它的形状”。如果 Ralph 循环对所有人都有效，讲者的判断是可以等待模型提供方把这类能力加入默认工具框架，而不必先在组织内部复制一遍通用基础设施。（来源时间：02:44:17–02:44:45）

这里的“等待”有明确边界：组织仍然可以试用和观察 Ralph 循环，判断它是否适合真实任务；需要被压缩的是围绕通用技巧持续打磨配置、脚本和仪式的时间。只要新工作流没有带来特定产品或代码库的独有洞见，它就更像公共能力的早期形态，而不是组织的长期护城河。

原则二的投入边界

如果某种工作流的收益来自普遍规律，先判断它是否会进入默认工具框架；如果收益来自特定用户、产品或代码库的事实，再为这些事实投入专门优化。原则二要求优化跟随独有信息，而不是跟随话题热度。

3.3 有效市场假说只是解释性类比

为了说明这个边界，讲者把它类比为有效市场假说（efficient market hypothesis）：当某个机会已经被广泛看见，单靠同一公共信息很难形成持续优势；对应到编程工作流，人人都能使用的通用技巧最终可能被默认工具吸收。（来源时间：02:44:44–02:44:55）

这一小节讨论的是类比的适用边界：有效市场假说只用来帮助理解公共能力为何可能被工具默认化。

这个类比指向公共能力容易被工具吸收的方向，并不是软件工程定律；工程判断仍应落回用户与代码库的具体事实。

3.4 真实业务优势信息不是金融收益指标

讲者随后解释，所谓真实业务优势信息（real alpha），是关于组织用户或代码库的某种信息，而模型可能并不知道这些信息。（来源时间：02:44:48–02:45:02）这一小节讨论的是专属工程信息的对象与作用：它指的是工程决策所需的专属上下文，例如产品必须服务什么场景、哪类交互是核心体验、现有代码库在哪些地方承受特殊约束。

不要把这个类比词读成金融或模型指标

这里的英文标签是讲者为工程判断借用的类比词，不是投资收益率、超额回报、模型能力分数，也不是可以代入公式计算的评分。它指向模型不知道的业务与代码库事实。

3.5 长对话 React 性能：值得优化的真实案例

Conductor 团队给出的例子是自己的聊天应用：产品必须很快地渲染很长的对话，因此性能对该应用很重要。团队愿意花大量时间优化 React 查询，使长对话更快显示，并接受代码库其他部分的取舍来服务这个核心约束。（来源时间：02:45:02–02:45:24）

这个案例说明了“真实业务优势信息”如何改变投入结论。长对话渲染是该产品的具体体验要求，模型可以知道通用的 React 写法，却未必知道这个应用为什么必须优先保障长对话性能。于是，针对 React 查询的工作流优化不再是追逐热门配置，而是把组织掌握的产品约束转化为实现优先级。

讲者还用“有一个漂亮的编辑器配置，却没有真正完成工作”来反向说明风险：工作流本身可以很精致，精致仍然不能替代交付。（来源时间：02:45:30–02:45:38）这句话不是对编辑器的评价，而是提醒读者检查优化对象是否真的连接到产品结果。

3.6 把原则转成工程投入判断

基于讲者的例子，读者可以在决定是否深度定制工作流前依次检查三件事：

1. 这项能力是否对大多数团队都成立，因而很可能成为默认工具的一部分？
2. 组织是否掌握模型不知道的用户、产品或代码库信息，使该优化具有明确的场景指向？
3. 优化是否直接服务真实交付中的关键体验，例如长对话的渲染速度，而不是只让工作流看起来更复杂？

实际投入判断仍应回到两个问题：通用能力是否会进入默认工具框架，专属约束是否直接影响用户体验或代码库结果。

3.7 本节小结

“不要试图打败市场”要求组织把工作流优化分成两类：普适技巧等待其进入默认工具框架，特定用户与代码库带来的真实业务优势信息则值得持续深挖。Ralph 循环展示了前一类，长对话 React 性能展示了后一类；有效市场假说在这里只是讲者的解释性类比，真实业务优势信息也只表示工程上的专属信息。下一节将把同样的投入边界推进到风险治理：哪些区域可以放松自动化，哪些区域必须保留严格人工审查。

4 原则三——建立人工审查保留区

本节的结论是：多编程智能体带来的速度，必须与风险分区一起设计。Conductor 的做法不是让所有代码都经过同一种审批强度，而是在高影响区域保留严格人工审查，在其他区域允许更松的自动化。讲者把前一种区域称为人工审查保留区 (slop-free zone)；结尾画面将第三条原则写作 ‘slop-free zones’，前文按画面校准。（来源时间：02:45:38–02:46:16）

4.1 人工审查保留区是风险分区

人工审查保留区指代码库或应用中必须进行严格人工审查的部分。讲者特别强调，Conductor 并非外界想象的“全量追求调用量”，也不是一味接受三万行变更请求；团队对某些代码区域非常谨慎，对另一些区域则保持宽松。（来源时间：02:45:44–02:46:14）

这个分区的价值在于把审查资源用在会改变系统结构、数据状态或长期行为的地方。讲者回顾，团队曾因没有认真保护这类区域而多次重写整个应用；这段经历是团队自身的经验性证据，不构成所有项目都会重写的普遍规律。（来源时间：02:46:14–02:46:30）

自动化成熟度体现在边界设计

人工审查保留区不是“禁止智能体修改任何文件”，也不是“整个代码库统一人工审批”。它要求团队先识别高影响区域，再把严格人工审查固定在这些区域；其余区域可以根据风险和收益采用更松的自动化。

4.2 migrations 的 CI 规则：把保留区落成硬约束

讲者给出的具体做法是：代码库有一个迁移文件（字幕中称为 `migrations`），持续集成（CI）规定，任何对该文件的修改都必须经过人工审查。（来源时间：02:46:30–02:46:42）这里的关键不在于文件名本身，而在于把“这个位置不能只依赖生成结果”转成团队成员都能看到的流程门槛。

这条规则形成了两层保护：CI 自动识别变更，人工判断变更是否会破坏数据迁移语义、顺序或长期兼容性。视频未展开 `migrations` 的具体格式、检查项或回滚机制。

4.3 治理保留区与内容质量判断：证据层级不能混用

本节两个相似表达需要分开处理。字幕中，讲者先明确定义前文括注的英文标签：它是代码库或应用中需要严格人工审查的区域，这是关于治理原则的直接口述证据。（来源时间：02:45:38–02:45:55）

随后讲者谈到 Slack 时，把人工写出的内容描述为“无敷衍”，并说明这些内容不是由 AI 写成的。（来源时间：02:46:41–02:46:48）这属于对人工书写内容的质量判断，和前面的保留区定义承担不同功能。

人工审查保留区与“无敷衍”需要分开理解：前者规定哪里必须由人审查，后者描述人工书写内容的质量判断；两者都不能仅凭相似发音或一张截图互相证明。

相似词不等于同一证据

“人工审查保留区”描述的是哪里必须保留人的审查；“无敷衍”描述的是讲者如何评价人工写出的组织内容。前者是代码与流程的治理边界，后者是内容来源的质量判断。二者都不能仅凭相似发音或一张画面截图互相证明。

4.4 Slack、CLAUDE.md 与 skills：高杠杆上下文也需要人工打磨

讲者把保留区的范围从代码扩展到组织上下文：Slack 中的人工书写内容、文档、规则入口文件以及 `skills`，都被团队投入大量时间打磨。（来源时间：02:46:41–02:47:07）具体文件名可以因项目而异；共同点是它们会反复影响人和智能体如何理解任务、代码库约束与团队习惯。

讲者用新人入职作比喻：如果每当一名新人开始工作时，都有机会在他耳边低声说一句话，团队会认真考虑这句话的内容；‘CLAUDE.md’、‘AGENTS.md’或 `skills` 所提供的上下文，就像每次智能体开始工作时加载进其上下文的长期耳语。（来源时间：02:47:05–02:47:37）这解释了为什么高质量规则文件值得人工维护：它们不是一次性说明，而是会重复进入后续工作的起点。

人工书写不等于永远正确；Slack、文档和 `skills` 仍需要持续审阅、更新与纠错。

4.5 从保留区回到整体 workflow

人工审查保留区与上一节的投入边界相互补充：上一节决定哪些通用 workflow 不值得过度定制，本节决定哪些高影响内容不能只交给自动化。两者共同形成分层策略：低风险、重复性高的部分可以让智能体快速推进；涉及数据迁移、组织规则和长期上下文的部分，需要把人的判断写入 CI、文档维护和人工审查流程。

风险等级、审批人数和阈值需要结合具体代码库、数据边界与团队责任制定。

4.6 本节小结

建立人工审查保留区，意味着把速度和审查强度按风险分开设计：迁移文件的 CI 规则提供了代码层面的硬门槛，Slack、规则入口文件、文档和 skills 则说明高杠杆组织上下文同样需要人工打磨。下一节将继续追问：当智能体需要理解组织真实工作方式时，这些上下文应当如何持续汇入可检索系统。

5 原则四：持续供给上下文——让组织事实成为智能体可检索的基础设施

本节的结论是：当编程智能体开始参与真实团队工作时，决定其组织效果的关键不只是模型能力，而是它能否持续获得这个组织正在讨论什么、遇到什么问题、如何做决定以及怎样协作的上下文。讲者把这条原则称为“持续供给上下文 (feed the beast)”：信息从团队日常工作流进入组织级集中上下文库，再通过查询工具向智能体开放。这个隐喻强调持续输入和可检索性，不表示应当无差别收集一切内部数据。

5.1 从启动规则到组织记忆

上一节已经说明，智能体每次启动都会加载高杠杆的规则文档；讲者用给新人耳语的比喻，说明这类文档需要被认真维护 (02:47:20–02:47:37)。本节把问题向前推进一步：静态规则只能告诉智能体“应该怎样工作”，组织上下文还需要让它知道“这个团队正在处理什么”。如果 Slack 消息、Discord 中的缺陷请求和会议讨论各自停留在原来的位置，智能体看到的就只是局部切片，无法建立对具体团队工作方式的连续认识。

这里的“组织上下文”不是泛泛的公司知识库，而是与当前工作有关的事实、讨论和决策痕迹。它回答的几个问题包括：某个问题是怎样被提出的，团队已经讨论过哪些方向，会议中形成了什么决定，某个缺陷请求属于怎样的产品语境。只有这些材料能够被稳定地汇入并检索，智能体才可能根据团队的真实工作方式行动，而不是只依据通用训练知识猜测。

本节的教学主张

组织上下文基础设施的作用，可以拆成两个连续环节：先把分散的工作事实汇入一个可查询位置，再给智能体一种能够取用这些事实的接口。前者解决“信息在哪里”，后者解决“智能体怎样找到它”；二者都不能单独等同于正确、完整或安全的答案。

5.2 CIA：把团队活动汇入一个集中位置

在 Conductor，讲者介绍了一个内部工具，称为 Conductor Internal Agent，并简称为 CIA (02:47:37–02:47:59)。CIA 是讲者对该内部工具的称呼；它被描述成组织级集中上下文库 (centralized database)：团队中不断发生的工作活动会被汇入同一个数据位置，供后续的智能体工作取用。

演示中的信息流由几个具体入口组成 (02:47:59–02:48:24)：

1. 有新的 Slack 消息时，CIA 会看到这条消息，并把它保存到 Postgres 表中。
2. 用户在 Discord 中提出缺陷请求时，讲者描述了同样的汇入路径。
3. 团队召开会议并进行录制后，会议内容也会进入 CIA 所代表的集中位置。

这组例子分别代表即时讨论、问题请求和较长的协作记录。智能体的组织上下文因此不再只是一份人工编写的启动说明，而是由团队运行过程持续产生的工作材料。讲者认为，如果智能体要在特定公司中有效，就应当尽可能了解“这个团队具体怎样工作” (02:48:21–02:48:40)。“尽可能”表达的是信息供给方向，不表示无限制采集或永久保留。



图 2：讲者展示的“数据库 + SQL tool”摘录，用来压缩说明 Feed the Beast 的信息集中与查询入口。²

5.3 SQL tool：把集中上下文变成可取用接口

讲者随后用一句高度压缩的建议概括这一机制：把信息放入数据库，再给智能体一个 SQL tool，让它处理后续工作 (02:48:40–02:49:00)。这句话的工程含义需要分层理解。

第一层是存储位置。数据库把原本分散在消息、缺陷请求和会议记录中的材料放到一个能够被统一查询的位置；这使得“组织记忆”具备了可定位的入口。第二层是取用接口。SQL tool 让智能体可以提出查询，从相关记录中寻找与当前任务有关的背景，再把这些背景带回任务上下文。第三

²视频画面时间区间：02:48:44–02:48:55。该画面是演讲者展示的二手摘录，不是 CIA 架构图。

层才是工作行动：智能体是否能正确理解记录、区分事实与意见、处理冲突并形成合适的下一步，仍然取决于具体任务和后续人工判断。

因此，SQL tool 不是“接入数据库就自动获得组织智慧”的保证。它更像一扇可查询的门：门把智能体带到材料所在的位置，却不负责替智能体证明材料完整、最新或相互一致。这个区分也解释了为什么“持续供给上下文”是一项基础设施原则，而不是一句关于模型会自动变聪明的宣传语。

把这条链条压缩成一个教学顺序，就是：来源事件进入 CIA，再进入组织级集中上下文库；智能体通过 SQL tool 查询相关记录，最后把查询结果带回具体任务。这里的箭头表示信息流和取用关系，不表示查询结果已经经过权限、隐私或质量审查。

来源入口	汇入的工作材料	本节能确认的作用
Slack (02:47:59–02:48:07)	即时团队消息	把正在发生的讨论放入集中位置
Discord (02:48:10–02:48:16)	缺陷请求	让问题请求进入同一上下文来源
会议录制 (02:48:14–02:48:21)	较长的协作与决策记录	扩大智能体可检索的组织背景
SQL tool (02:48:40–02:49:00)	对集中位置发起查询	提供取用接口，不保证结论质量

5.4 部署时仍需补足治理条件

真实部署还需补足权限、隐私、数据保留、审计、敏感信息过滤、数据质量与错误恢复；这些治理机制不由数据库或 SQL tool 自动提供。

不要把隐喻读成无限采集

“持续供给上下文”要求信息可被智能体取用，仍需遵守组织的权限、隐私与数据质量边界。

5.5 本节小结

“持续供给上下文”的核心不是给智能体更多抽象知识，而是把团队真实工作中产生的讨论、问题和会议材料持续汇入可查询的组织级集中上下文库，再通过 SQL tool 提供取用入口。CIA 是讲者给出的内部案例，支持的是“集中信息供给有助于特定团队工作”这一原则；它没有证明权限、隐私、审计和数据质量已经被解决。下一节将把问题从“智能体知道什么”转向“智能体在哪里持续运行以及怎样与团队协作”。

6 原则五：自由运行智能体——把执行从笔记本寿命中解放出来

本节的结论是：随着智能体能够运行更长时间、承担更困难的任务并与更多人和智能体协作，单机终端会成为工作流的能力上限。讲者提出“自由运行智能体 (free-range agents)”，所指的是具有持久沙盒、能够探索和协作的执行空间；演示进一步展示了从 Git 工作树迁移到云端沙盒、关闭笔记本后继续运行、实时协作工作区，以及通过外部消息和 Conductor API 创建新工作区的产品路径。这里描述的是持久执行和协作接口，不能扩写成无限权限、无人监督的自动部署或智能体已经复制自己。

6.1 定义：自由运行不是无边界权限

讲者把自由运行智能体定义为一种运行方式：给智能体足够空间探索代码库、处理困难任务，并让它在关闭笔记本后仍能继续；同时，还要给它与其他智能体和人协作、产生更多工作机会的方式 (02:48:55–02:49:30)。这一定义包含三个互补条件：执行不被本地终端寿命截断，任务有空间进行探索，工作结果能够回到团队协作中。

这里的“自由”描述运行空间和持续时间，权限仍需单独定义；自由运行智能体首先是一种工作空间设计原则。

讲者还说，希望智能体有机会“产生更多的自己” (02:49:15–02:49:30)。结合后面的 API 演示，这句话可理解为派生更多工作空间或启动更多任务的产品设想，不表示一个智能体已经完成自我复制，也不表示系统会无条件创建任意数量的智能体。

读者应保留的三层含义

自由运行智能体首先需要持久执行空间，其次需要团队可见的协作界面，最后才可能在受控接口上派生后续工作。三层分别回答“能否继续运行”“人和其他智能体能否接入”“能否通过外部请求启动新工作”；任何一层都不能被偷换成无限制行动能力。

6.2 从 Git 工作树到云端沙盒

讲者随后展示了 Conductor 的一个新版本，并说明它围绕云端协作展开；每个工作区顶部有云图标，点击后可以查看智能体正在运行的沙盒信息 (02:50:10–02:50:42)。这个画面把“沙盒”从抽象概念变成了可被用户观察的工作空间入口。

接着，讲者明确对比了两种运行载体：在演讲当周之前，Conductor 中的任务基本建立在 Git 工作树 (git worktree) 上；演讲时则转向云端沙盒 (cloud sandbox) (02:50:42–02:50:55)。Git 工作树主要把代码工作隔离在本地仓库结构中，而云端沙盒把任务的运行寿命从本地笔记本中移开；这里描述的是演讲时的产品状态。

最直接的演示结果是：用户关闭笔记本后，智能体仍会继续运行 (02:50:40–02:50:49)。这个事实支撑“持久执行”这一能力边界；它没有说明任务一定成功、结果一定安全，或系统能够在无人查看时自动完成任何生产变更。

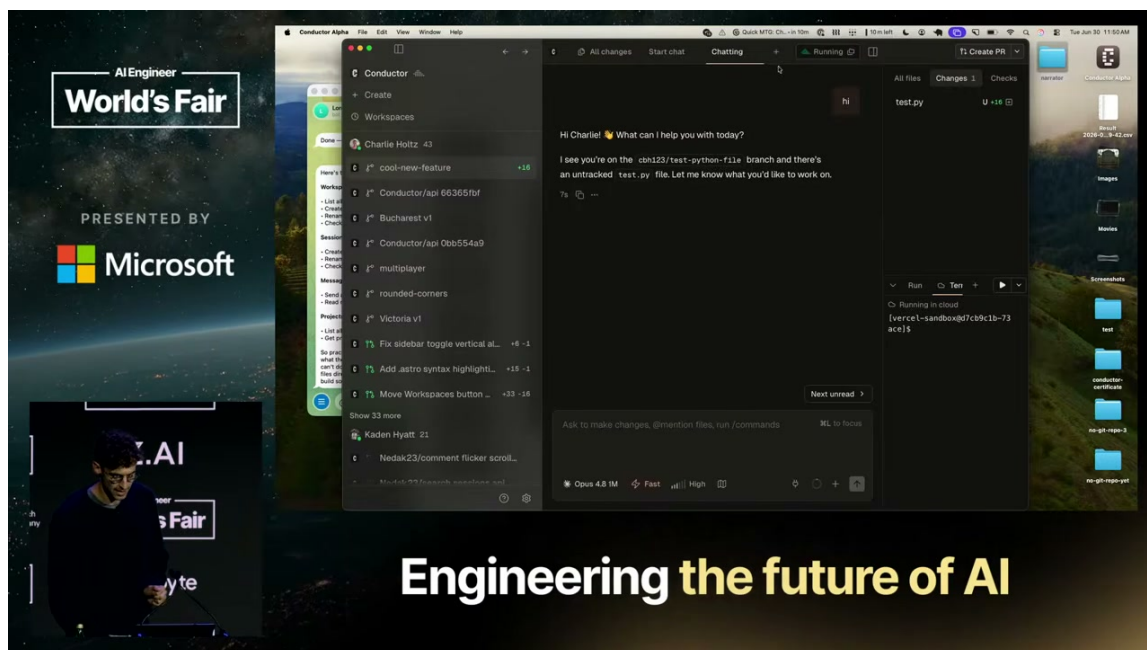


图 3: Conductor 工作区中的云端运行状态与 workspace 列表。³

6.3 实时协作工作区：从个人任务到团队状态

云端运行之后，讲者把第二个重点放在实时协作工作区（collaborative workspace）上。演示界面先显示讲者自己的工作列表，再显示团队成员正在处理的工作；他可以进入同事的工作区，查看对方已经完成的修改，并留下关于制表符或空格的审查消息（02:51:00–02:52:25）。对方能够在同一个工作区中看到消息并继续聊天，界面还显示对方正在输入（02:52:25–02:52:43）。

这个协作模型改变了观察对象：人不必只通过最终提交物猜测智能体或同事做了什么，而可以进入共享状态，查看正在进行的工作并提供实时反馈。讲者在演示中把它总结为团队成员可以实时共享同一个协作工作区（02:53:00–02:53:13）。演示直接展示了查看、审查、消息和共享状态，没有展示完整的权限同步、审计系统或质量保证。

讲者还给出一个为什么需要协作的组织理由：伟大的成果通常由团队完成，而模型变强后，团队能够建设更有野心的东西，因此需要更多人和更多智能体一起工作（02:51:28–02:52:01）。这使协作不只是一个聊天功能，而是长时运行和任务规模增长后的配套接口。运行空间扩大以后，团队也需要知道谁在做什么、怎样进入工作、在哪里给出反馈。

³视频画面时间区间：02:50:10–02:50:55。

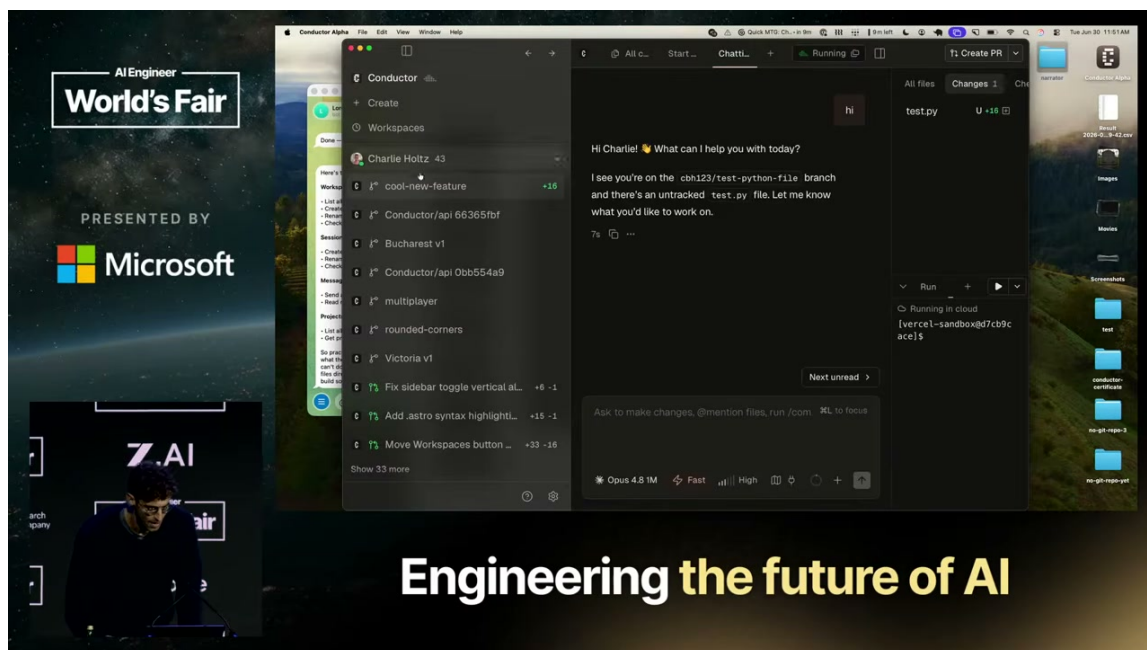


图 4: 共享工作区中可观察团队成员、工作状态和云端运行信息。⁴

6.4 外部消息如何启动一个工作区

演示的最后一层把工作区入口移到了 Conductor 外部。讲者称，云端和自由运行沙盒可以通过 API 让智能体派生工作；随后打开一个名为 Lord Crandon 的 OpenClaw 实例，说明它能够访问 Conductor API (02:53:37–02:54:01)。OpenClaw 是现场演示中的具体实例，Conductor API 是演示所使用的外部调用接口；这条演示路径不等于通用自动化标准。

讲者接着描述了请求路径：无论人在手机、Telegram、Slack 或其他消息入口，都可以向这个 OpenClaw 实例提出请求，让它创建一个新的工作区，并让新工作区把按钮改成蓝色 (02:53:59–02:54:17)。随后，讲者把消息发送给 Lord Crandon；由于该实例能够访问 Conductor API，它可以启动这项工作 (02:54:17–02:54:27)。

画面后续显示工作区已经创建，智能体开始在其中工作；讲者说自己离开时，工作区仍在设置，任务会继续进行 (02:54:30–02:54:45)。因此，这个演示具体证明的是一条产品想象中的入口链：外部消息请求 → OpenClaw 调用 Conductor API → 创建工作区 → 智能体在其中继续工作。它没有证明 API 默认拥有任意权限，也没有证明智能体已自我复制，更没有提供失败恢复、审批或生产部署的机制。

讲者随后把这类能力概括为可以在自由运行智能体之上继续构建更多产品 (02:54:45–02:54:52)，说明持久工作区可以成为其他功能的底座。

讲者还称，协作功能将在演讲当周向所有 Conductor 用户推出，并认为协作会成为当年的重要新界面变化 (02:53:19–02:53:37)。该判断属于演讲当周的产品发布语境。

⁴视频画面时间区间：02:51:00–02:53:13。

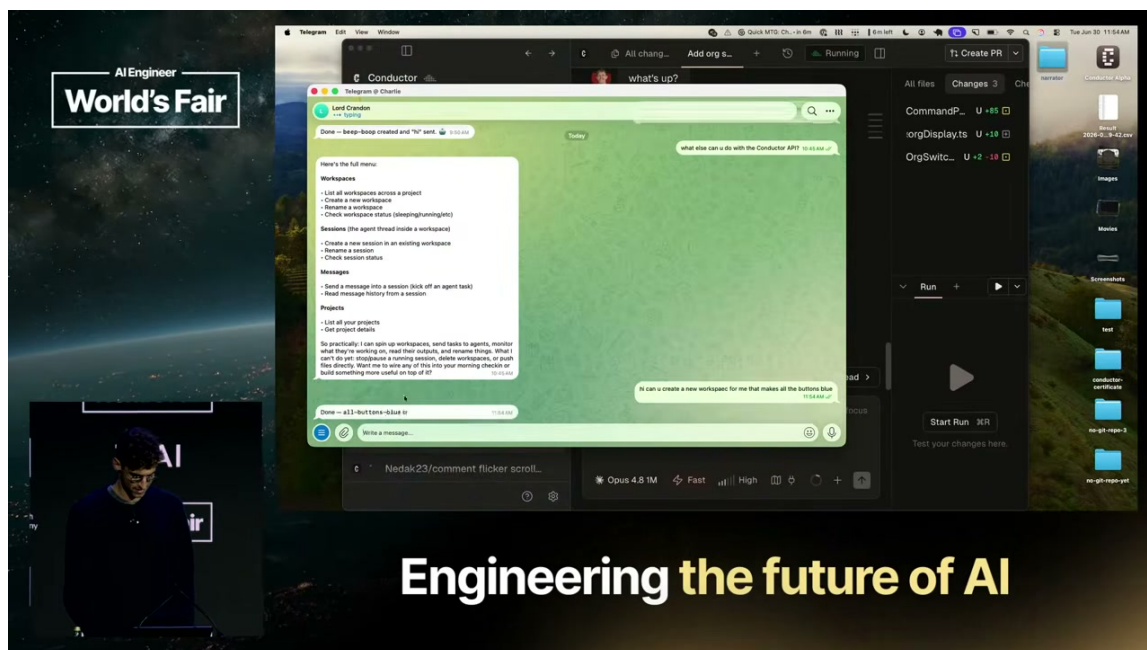


图 5: OpenClaw 通过 Conductor API 请求创建工作区的现场演示。⁵

6.5 部署时仍需补足治理条件

持久沙盒、共享工作区和外部 API 入口仍需配套身份、隔离、审批、审计、回滚与成本控制。

三条禁止推断

关闭笔记本后继续运行不等于任务必然成功；派生工作不等于自我复制；访问 API 也不等于无限权限。

6.6 本节小结

自由运行智能体把执行空间从笔记本寿命中解放出来：云端沙盒承载持久任务，实时协作工作区让人和团队成员进入共享状态，外部消息与 Conductor API 则展示了创建新工作区的入口。下一节将把这些基础设施收束到最终的人机关系：人处于创造中心，由多个智能体组成乐团式协作。

7 原则六：乐团式协作，而不是工厂式流水线

本节的中心判断是：AI 工具的长期目标不应只被理解为扩大自动化产量，还应当让人处于中心 (human at the center)，以乐团式协作 (orchestras, not factories) 的方式对一组智能体进行方向、节奏和细节层级的编排。讲者借“乐团”与“工厂”的差异，给出了前五条工程原则的体验层收束。

⁵ 视频画面时间区间：02:53:37–02:54:45。画面支持外部消息到工作区创建的路径，不支持推断无限权限。

本节主张

“乐团，而不是工厂”描述的是人机关系的目标形态：人保留方向、取舍与整体判断，智能体承担可并行、可持续、可协作的执行。工具的价值由此体现为扩大人的创造流，而非把人变成只负责按按钮的流水线经理。

7.1 问题：自动化提高产量，却可能改变人的位置

讲者承认工厂隐喻具有一部分吸引力。工厂让人联想到自动化、效率和更大产量；自动化确实能够让生活更高效，也能让团队创造更多东西（02:54:54–02:55:29）。因此，问题不在于自动化本身，而在于组织是否把未来只想象成一条持续吐出 feature 的流水线。

讲者明确表示，自己不喜欢把新工具称为“software factories”，因为这个词容易把注意力引向重复生产与产量管理。若人只负责分派任务、推动智能体生成下一个 feature，再像工厂线经理一样检查按钮状态，人的创造性判断就会被推到过程之外（02:55:02–02:56:12）。这也是本节与前五条原则的连接点：持久沙盒、上下文数据库和协作接口最终都要回答“人如何继续参与创造”。

7.2 核心画面：人挥动指挥棒，智能体团队展开工作

讲者给出的正面意象是站在乐团前方挥动指挥棒。指挥动作朝向不同方向时，不同的智能体团队开始工作；人可以在需要时放大细节，也可以在大多数时间缩小视野，保持对全局的判断（02:55:34–02:55:58）。

这套比喻包含三个可操作的界面要求：

1. **方向可表达**：人能够说明想解决的问题、希望达到的体验和当前优先级。
2. **执行可编排**：智能体可以并行工作，工作状态与协作关系对人可见。
3. **细节可缩放**：人能够在整体目标与某个具体修改之间自由切换，而不必一直守在每个任务的按钮旁。

“挥动指挥棒、zoom in、zoom out”分别对应方向控制、局部审查和全局观察；共同目标是让人保留判断密度，同时把执行规模交给多个可观察的工作单元。

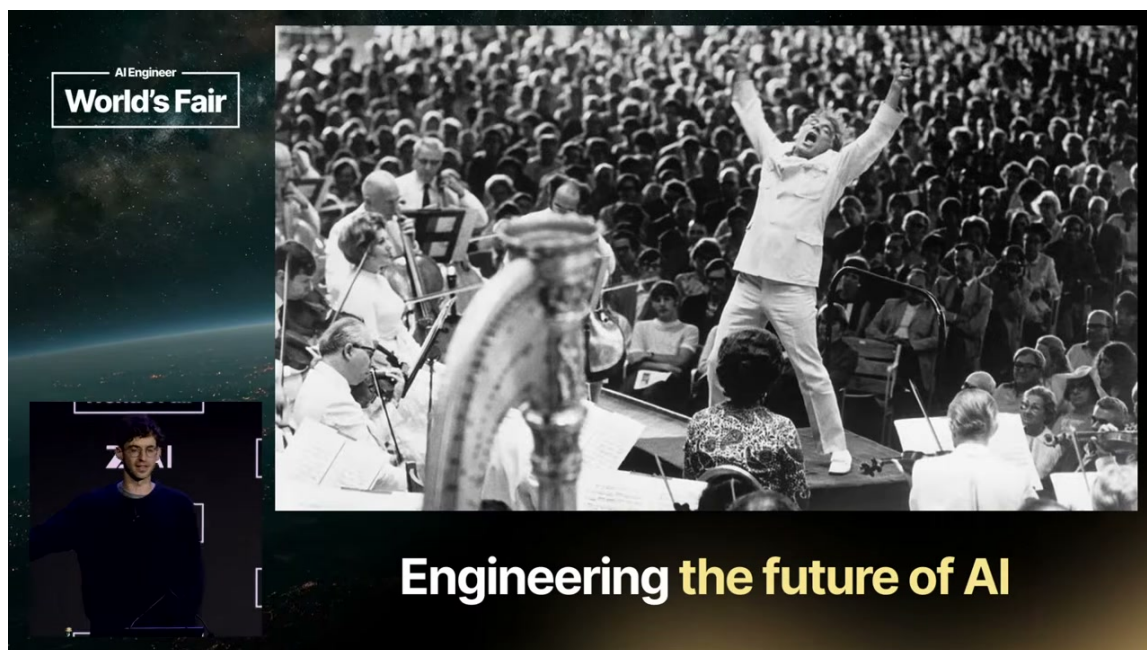


图 6: 讲者用于说明“乐团，而不是工厂”的指挥家与乐团视觉隐喻。⁶

7.3 为什么工厂式 feature 管理不足以承载新工作方式

讲者把“功能工厂式生产 (feature factories)”作为过去已经出现过的组织想象：团队像流水线一样不断推动下一个功能，管理者负责让生产过程持续运转 (02:56:00–02:56:12)。这类模式的限制在于，它把成功简化为产量，把人的位置简化为调度与监督，而没有回答软件是否具有人类设计感、是否解决了重要问题、是否让创造者处于工作流之中。

在讲者的愿景中，软件应当具有人类参与和精心制作的感觉。使用者希望与优秀的人类同事和 AI agents 处于同一创造空间，既能够感受到能力与乐趣，也能够关键时刻承担取舍责任 (02:56:12–02:56:58)。因此，“工厂”并非被简单判定为低效；它被指出为一种会把人的体验和责任压扁的组织隐喻。

隐喻的边界

“乐团”不表示每个智能体都具有人的创造责任，也不表示团队可以取消审查和治理。“工厂”也不等于所有自动化都没有价值。两种词汇用于提醒读者：自动化规模与人的创造位置需要同时设计，前五条原则提供了运行、上下文和审查的基础条件。

7.4 工具建设者的责任：让人感觉处于其中

讲者把责任进一步扩展到工具建设者：既然新一代工具由今天的工程团队构建，建设者就需要有意识地选择能让人感到兴奋、能干、处于创造流中 (in the flow) 的界面与词汇 (02:56:18–02:56:34)。这里的“感受”不是装饰性体验指标，而是对工作关系的设计判断：人在工具中看到是自己的意图正在展开，还是一组需要逐个催促的黑箱任务。

这条责任命题与 Conductor 的实时工作区和云端沙盒相连。共享状态让人能够观察团队正在做什么，持久运行让人不必守在笔记本旁，外部 API 让人可以从消息入口启动工作。若这些能力最终

⁶视频画面时间区间：02:55:34–02:56:58。该画面是概念隐喻，不是软件架构图。

只增加了按钮数量，却降低了人的整体理解，它们就会重新滑向工厂式管理。若它们让人能够保持方向、缩放细节并享受创造过程，则更接近讲者希望的乐团式协作。

7.5 本节小结

“乐团，而不是工厂”是全段的价值收束。工厂隐喻提醒人们自动化可以提升效率和产量；讲者进一步要求工具让人处于中心，让人以指挥者的方式提供方向、切换尺度、观察并取舍。智能体的并行执行、持久沙盒和实时协作因此应当服务于创造流，而不应把人压缩成流水线经理。下一节将把六条原则合并为可迁移的工作系统与检查表。

8 总结与迁移：把 Stickfo 组合成一套工作系统

本节的中心判断是：六条原则只有被组合成一套互相约束的工作系统时，才会把并行执行转化为人的创造能力。单独追逐前沿会陷入工作流炫技，单独扩大智能体数量会增加协调负担，单独集中信息又可能放大治理风险；Stickfo 的价值在于把发现、投入、审查、上下文、运行空间和人机关系串成一条完整链路。

8.1 讲者的结尾：从六条原则回到 Stickfo

在收束部分，讲者重新列出六条原则：Stay near the frontier; Don't try to beat the market; Create slop-free zones; Feed the beast; Free-range agents; Orchestras, not factories (02:57:00–02:57:21)。讲者称 Stickfo 是帮助记忆这组原则的助记词；演讲没有给出可靠的逐字母展开。



图 7: 演讲结尾完整展示的六条原则；画面文字校准第三条为 ‘slop-free zones’。⁷

8.2 六条原则的系统链条

把讲者的六条原则放在同一张工作系统中，可以得到以下递进关系：

⁷ 视频画面时间区间：02:57:00–02:57:21。

1. **靠近技术前沿：**让组织尽早接触新能力，以发现新的工作方式和产品机会。
2. **不要试图打败市场：**用“为什么它还不是默认能力？”限制对通用工作流的长期定制投入。
3. **建立人工审查保留区：**把数据迁移、长期规则和高杠杆上下文放入必须由人严格审查的区域。
4. **持续供给上下文：**把讨论、缺陷请求和会议记录汇入可检索的组织级集中上下文库。
5. **自由运行智能体：**用持久、可观察、可协作的云端沙盒承载长时任务与多方协作。
6. **乐团式协作：**让人保持方向、取舍与整体视野，使执行规模服务于创造流。

前五条原则提供运行系统的条件，第六条原则规定这些条件最终应当支持怎样的人机体验。它也解释了为什么“组织上下文”与“云端沙盒”必须同时出现：智能体既要知道团队如何工作，也要有时间和空间把这些信息转化为可观察的行动。

一句话压缩

先靠近前沿发现机会，再用市场启发式控制投入；把高风险区域交还给人工审查，把组织事实供给给智能体；让智能体在持久协作空间中运行，最终让人以指挥者而非流水线经理的方式保持创造中心。

8.3 迁移检查表：把原则变成设计问题

下表把六条原则转成组织内可以讨论的设计问题。

编号	设计问题	需要观察的证据或边界
1	组织是否有人持续使用新工具并把经验带回团队？	是否出现真实任务中的新洞见，而不只是工具试用记录。
2	当前工作流优化是否依赖模型未知的用户或代码库事实？	若收益对所有团队都普遍成立，应优先等待默认能力；若收益来自专属瓶颈，才值得深挖。
3	哪些代码、数据迁移和规则文件必须强制人工 review？	是否已经通过 CI、文档维护或责任人制度写出明确边界。
4	智能体能否检索团队真实讨论、会议和缺陷上下文？	是否同时考虑权限、隐私、数据质量和过期上下文；需结合组织实际设计。
5	智能体能否在持久、可观察的沙盒中运行并与人协作？	是否能查看工作状态、进入共享工作区并限制外部 API 触发范围。
6	使用者是否仍能保持整体方向、细节缩放和创造乐趣？	工具是否帮助人处于创造流中，还是把人变成只负责催促下一个功能的 line manager。

权限、审计、成本、失败恢复等部署细节仍需结合组织实际补足。

8.4 讲者愿景与教学归纳的分层

讲者在最后的愿景中说，不希望进入一座黑暗的工厂，也不希望成为流水线经理；讲者希望自己像 Steve Jobs 与一组优秀的人类和 AI agents 一起设计 Mac 一样，处于共同的创造空间中 (02:56:34–02:57:04)。这段话表达的是一种人处于中心的工具愿景。

迁移时可以沿三层组织：前沿试验连接真实工作；自动化系统同时设计投入边界、人工保留区、组织上下文和持久运行空间；所有基础设施回到人的方向感、判断力与创造流。

不要把 Stickfo 当成万能配方

Stickfo 是一组经验性原则和记忆口号，不是完整的组织架构、权限规范或成功保证。它没有替代风险评估、数据治理、审计、成本控制和失败恢复；它提供的是一张帮助团队讨论“如何成为最快构建者”的问题地图。

8.5 本节小结

这场 talk 的可迁移答案可以压缩为：让前沿试验带来真实洞见，用市场启发式节制通用优化，把关键区域交还给人工审查，把组织上下文变成可检索基础设施，再给智能体持久而可协作的运行空间；最终让人保持整体指挥与创造流。Stickfo 由此构成一套从发现到治理、从信息到运行、从执行到人机体验的工作系统。